

CS 6903 - Network Security

PART 1: DNS BASICS

1. Internet Names and Addresses

The internet needs two kinds of identifiers for machines:

- **Addresses (e.g., 129.10.117.100):** Machine-readable labels that conform to the structure of the network. Computers use these to locate and communicate with each other.
- **Names (e.g., www.iith.ac.in):** Human-readable labels that conform to organizational structure. Easier for people to remember and use.
- **Domain Name System (DNS):** The system that maps human-readable names to machine-readable IP addresses.

2. History: Before DNS

Before DNS existed, all name-to-IP mappings were stored in a single file called hosts.txt.

- **Location:** /etc/hosts on Linux, C:\Windows\System32\drivers\etc\hosts on Windows.
- This was a centralized, manual system. Changes were submitted to SRI (Stanford Research Institute) via email, and machines periodically downloaded new copies via FTP.
- Administrators could pick any name they wanted with no restrictions (e.g., alans_server_at_neu_pwns_joo_lol_kthxbye).

Why the hosts.txt system failed:

- Not scalable — SRI could not handle the increasing load as the internet grew.
- Hard to enforce uniqueness — for example, 'MIT' could refer to Massachusetts Institute of Technology or Melbourne Institute of Technology.
- Many machines had stale or inaccurate copies of hosts.txt. This led to the creation of DNS.

3. DNS at a High Level

DNS is a Distributed Database system with the following characteristics:

- **No centralization:** There is no single server that holds all DNS records. The data is spread across many servers worldwide.
- **Client/Server Architecture:** Clients (your computer) send queries; servers respond with answers.

- **Protocol:** Uses UDP port 53 by default. Some implementations also use TCP (for large responses or zone transfers).
- **Hierarchical Namespace:** Names are organized in a tree structure (e.g., .com → google.com → mail.google.com), replacing the original flat namespace.

4. Naming Hierarchy

DNS names are organized in a tree-like hierarchy:

- **Top Level Domains (TLDs):** Sit at the top of the hierarchy — e.g., .com, .edu, .gov, .org, .net, country codes like .uk, .fr, etc.
- **Maximum tree depth:** 128 levels.
- **Each Domain Name is a subtree:** Example: .edu → neu.edu → ccs.neu.edu → www.ccs.neu.edu.
- **Name Collisions Avoided:** Because domains are organized under TLDs, neu.com and neu.edu are two completely different, unique domains.

5. Hierarchical Administration

The DNS tree is divided into zones, and each zone has an administrator responsible for that portion of the hierarchy.

- Example: CCIS controls *.ccs.neu.edu (all subdomains of ccs.neu.edu).
- NEU controls *.neu.edu (all subdomains of neu.edu).
- Verisign manages .com and .net TLDs.
- ICANN administers the Root Zone.

6. Server Hierarchy

DNS servers are organized hierarchically, each with a specific role:

- **Authority over a zone:** Each DNS server holds records only for its own zone — it does not need to store all DNS names in existence.
- **Zone storage:** Stores all records (name→IP mappings) for hosts and domains within its zone. Records may be replicated for robustness.
- **Root server knowledge:** Every DNS server knows the addresses of the root servers, which allows it to resolve any unknown name by starting from the top.
- **Root servers:** Know about all TLDs. They are the ultimate authority when no other server can answer.

7. Root Name Servers

Root servers are responsible for the Root Zone File, which lists all TLDs and who controls them.

- **Size:** ~272 KB. Contains NS records pointing to TLD servers (e.g., com. → a.gtld-servers.net.).
- **Administered by:** ICANN (Internet Corporation for Assigned Names and Numbers).
- **13 root servers total**, labeled A through M. 6 of these are anycasted — meaning they are globally replicated across many physical machines sharing the same IP address.
- They are contacted when names cannot be resolved by any cached data. In practice, most systems cache root zone information so queries rarely reach the root servers directly.

8. Local Name Servers

Every ISP or organization has a local, default name server.

- **Configuration:** Usually configured automatically via DHCP when you connect to a network.
- When your computer wants to resolve a domain name, it first sends the query to this local name server.
- **Caching:** Local name servers frequently cache query results. This speeds up future lookups and reduces load on higher-level servers.

9. Authoritative Name Servers

An authoritative name server stores the actual name-to-IP mapping for a specific domain.

Example of how resolution works for `www.neu.edu`:

- Your local DNS server first asks the Root server.
- The Root server redirects to the `.edu` authority server.
- The `.edu` server redirects to the `neu.edu` authority server (Northeastern's DNS).
- The `neu.edu` server has authority for `neu.edu` and responds: `www.neu.edu = 155.33.17.68`.

10. Basic Domain Name Resolution

The resolution process works as follows:

- Every host knows a local DNS server and sends all queries to it.
- **If the local DNS can answer directly:** Either because it is also the authoritative server for that name, or because it has a cached copy of the record.
- **If not:** It navigates down the DNS hierarchy to find the authoritative name server.
- Every local DNS server knows the root servers, so it can always start the search from the top.

- Caching is used to skip steps whenever possible (e.g., skip root and go directly to .edu if the root zone file is already cached).

11. Recursive DNS Query

In recursive resolution, the full burden of resolving the name is placed on the first contacted name server.

- The client sends a query to its local DNS server and waits for a complete answer.
- The local DNS server contacts the root, then the TLD server, then the authoritative server — doing all the legwork.
- **Tracking queries:** Random IDs are embedded in DNS queries so the server knows which client to forward the response to.
- **Trade-off:** Keeping state in the network adds complexity and resource usage on the DNS server side.

12. Iterated DNS Query

In iterative resolution, each server simply points the client to the next server that might know the answer.

- The contacted server replies: 'I don't know this name, but try this other server.'
- The local DNS server iteratively contacts each suggested server until the answer is found.
- **Current standard:** This is how DNS actually works today. The local resolver does the iterating, not the root or TLD servers.

13. DNS Resource Records

DNS queries specify a name and type. The response is a Resource Record (RR) with four fields:

- **(name, value, type, TTL):** name = what you're looking up, value = the answer, type = what kind of record, TTL = how long to cache it.
- Multiple records can be returned for a single query.
- The meaning of name and value depends on the record type.

14. DNS Record Types

Type A / AAAA

- **Name:** Domain name (e.g., www.ccs.neu.edu)
- **Value:** IP address (e.g., 129.10.116.81)
- **A = IPv4, AAAA = IPv6.** Example: Query Type A for www.ccs.neu.edu → returns 129.10.116.81.

Type NS (Name Server)

- **Name:** Partial domain (e.g., ccs.neu.edu)
- **Value:** Name of the DNS server responsible for that domain (e.g., 129.10.116.51)
- Tells the resolver: 'Go send your query to this other server.'

Type CNAME (Canonical Name)

- **Name:** Alias hostname (e.g., foo.mysite.com)
- **Value:** The canonical (real) hostname (e.g., bar.mysite.com)
- Used for aliasing — one canonical name can have many alias names. CDNs use CNAME extensively.

Type MX (Mail Exchange)

- **Name:** Domain in an email address (e.g., ccs.neu.edu)
- **Value:** Canonical name of the mail server (e.g., amber.ccs.neu.edu)
- Used to route email to the correct mail server for a domain.

15. DNS as an Indirection Service

DNS does much more than just make names easier for humans to remember. It provides powerful indirection:

- **Changing IPs becomes trivial:** If you want to move your web server to a new machine with a different IP, just update the DNS record. Users continue to use the same name without noticing the change.

16. Aliasing and Load Balancing

DNS enables two important capabilities:

- **One machine can have many aliases:** Multiple domain names (e.g., www.reddit.com, www.foursquare.com, *.blogspot.com) can all point to the same physical server.
- **One domain can map to multiple machines:** A single domain (e.g., www.google.com) can resolve to many different IP addresses, distributing traffic across multiple servers for load balancing.

17. Content Delivery Networks (CDNs)

DNS is the backbone of how CDNs work:

- DNS responses can vary based on the user's geographic location and ISP.

- A user in the US asking for netflix.com gets routed to a nearby US server; a user in Europe gets routed to a European server.
- This reduces latency and improves content delivery performance by keeping traffic geographically close to the user.

PART 2: DNS SECURITY

18. The Importance of DNS

19. Denial of Service (DoS) Attacks on DNS

Attackers can flood DNS servers with fake requests until the servers fail and cannot respond to legitimate queries.

- **October 2002:** A massive DDoS attack targeted all 13 root name servers simultaneously.
- **Effect:** Users barely noticed because the root zone file is cached almost everywhere. The cached data continued to serve resolution requests.
- **More targeted DoS attacks can be effective:**
 - Attacking a local DNS server disables DNS access for all users on that network.
 - Attacking an authoritative server makes a specific domain unreachable.

20. DNS Hijacking

DNS hijacking redirects DNS lookups to malicious servers. There are several methods:

- **Malware/Trojan infection:** Infect the victim's OS or browser. Many trojans modify entries in /etc/hosts so that *.bankofamerica.com resolves to a fake IP (evilbank.com), bypassing DNS entirely.
- **Man-in-the-Middle (MITM):** An attacker intercepts communication between the user and the DNS server, replacing legitimate answers with malicious ones.
- **Response Spoofing:** The attacker eavesdrops on DNS requests and sends a fake response before the legitimate server can reply, 'outracing' the real answer.

21. DNS Spoofing

DNS spoofing raises the fundamental question: How do you know that a given name-to-IP mapping is correct?

- A user asks: 'Where is bankofamerica.com?' and expects the real IP (e.g., 123.45.67.89).
- An attacker (dns.evill.com) intercepts the query and returns a fake IP (66.66.66.93) pointing to their malicious server.

- The victim's browser connects to the malicious server, thinking it's the real bank. Credentials and data are stolen.

22. DNS Cache Poisoning

Cache poisoning is a more severe and wide-reaching attack than simple spoofing.

How it works:

- An attacker sends a fake DNS response to a local DNS server (e.g., dns.neu.edu) while it is waiting for a legitimate reply.
- The fake response contains a malicious IP for a domain (e.g., bankofamerica.com = 66.66.66.92).
- The local DNS server caches this poisoned record.
- **Impact:** Until the TTL (Time to Live) expires, every user who queries dns.neu.edu for bankofamerica.com receives the poisoned, malicious IP address.
- **Scale:** Unlike spoofing a single user, cache poisoning affects entire ISPs — potentially thousands or millions of users with a single poisoned cache entry.

23. Solution: DNSSEC

DNSSEC (DNS Security Extensions) solves DNS hijacking and spoofing by cryptographically signing DNS records.

- **Core idea:** Resource records are signed with cryptographic keys. The resolver can verify these signatures to ensure the response has not been tampered with.

Two new record types introduced by DNSSEC:

- **DNSKEY:** Name = Zone domain name. Value = The public key for that zone. Used by resolvers to verify signatures.
- **RRSIG:** Name = (type, name) tuple — the query itself. Value = Cryptographic signature over the query results. The resolver uses this to confirm authenticity.

24. DNSSEC Hierarchy of Trust

DNSSEC creates a chain of trust from the root down to individual domains:

- Each zone's keys are signed by the parent zone's keys (e.g., .com's keys are signed by the root).
- If a malicious server (dns.evil.com) returns a fake IP for bankofamerica.com, it cannot produce a valid cryptographic signature because it does not have the private key of the legitimate bankofamerica.com zone.
- The resolver checks the signature against the public key published in DNSKEY records, following the chain: Root Zone (ICANN) → .com (Verisign) → bankofamerica.com.
- A missing or invalid signature causes the resolver to reject the response, protecting users from hijacking and spoofing.